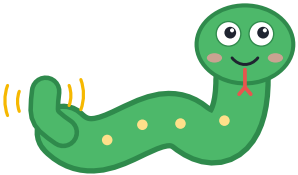
## Recitation 4

Part 4: Data Science and AI

# Recitation 4: Data Science and AI

### Your own classes, arrays, tables, plots and a first model



Dror Jacoby
Tel Aviv University

# Where we are


L1
Hello, Python!
L2
Variables & Types
L3
Input & Output
L4
Making Decisions
L5
Lists & Sets
L6
Loops
L7
Functions
L8
Dicts & Debugging
L9
Objects, NumPy & Pandas
L10
Plots & ML
Today we practise L9–L10

# Today's plan

**◎ By the end of today you can…**

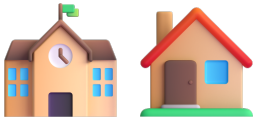
- Build a small class and use its methods
- Do math on a whole array at once
- Load a table, pick columns, filter rows
- Pick the right plot and train a first model



Last recitation: everything here feeds your final project. 📈

# How recitations work

| | |
|---|---|
| in class | Recap, warm-up, worked problems, and the exercises marked **IN CLASS** |
| at home | Exercises marked **AT HOME**: practice before the project |
| solutions | At the end of this deck. Try first, peek later! |



There is more here than one session. That is on purpose.

Topic 1

# Recap and warm-up

Recitation 4 • Classes, NumPy, Pandas, plots & ML



TODAY

1 **Recap and warm-up**

2 Worked problems

3 Your turn

4 Exam corner

5 Solutions

# Lecture 9 in one slide: classes

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_to(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

home = Point(0, 0)
print(home.distance_to(Point(3, 4)))
```

OUTPUT

```
5.0
```

| Word | Meaning |
|---|---|
| class | the cookie cutter (a type) |
| object | one cookie: `Point(0, 0)` |
| `__init__` | runs when an object is made |
| `self` | "this object" |
| attribute | data: `home.x` |
| method | action: `home.distance_to(...)` |

Every object keeps **its own** data.

# Lecture 9 in one slide: NumPy

| Write | Result |
|---|---|
| `import numpy as np`<br>`p = np.array([12, 15, 9])`<br>`p * 2`, `p + 1`<br>`p > 10`<br>`p[p > 10]`<br>`np.mean(p)`, `np.median(p)`, `np.max(p)` | open the toolbox, nickname `np`<br>an array: a list built for math<br>`[24 30 18]`, `[13 16 10]`<br>`[ True True False]`<br>keep the True ones: `[12 15]`<br>`12.0`, `12.0`, `15` |
| `g.shape`, `g[1, 2]`<br>`g.mean(axis=1)`, `g.mean(axis=0)` | (rows, columns), row 1 column 2<br>per row, per column |



Math on the array = math on every item.

# Lecture 9 in one slide: Pandas

| Write | Means |
|---|---|
| `import pandas as pd` | the table toolbox |
| `df = pd.DataFrame({...})` | a table from a dictionary |
| `df = pd.read_csv("file.csv")` | a table from a CSV file |
| `df.head()` | the first 5 rows |
| `df["price"]` | one column |
| `df["price"].sum()`, `.mean()`, `.median()` | numbers about a column |
| `df[df["drink"] == "Tea"]` | only the rows that answer True |
| `df["drink"].value_counts()` | how many of each |
| `df.describe()` | the whole summary at once |



A spreadsheet inside Python.

# Lecture 10 in one slide: plots and models

## Plots 📈

| | |
|---|---|
| `plt.plot(x, y)` | change over time |
| `plt.bar(names, values)` | compare groups |
| `plt.scatter(x, y)` | one goes with another? |
| `plt.hist(values)` | how values spread |

Then `title`, `xlabel`, `ylabel`, `legend`, `show()`.

## Machine learning 🤖

| | |
|---|---|
| **1. Create** | `model = LinearRegression()` |
| **2. Fit** | `model.fit(examples, answers)` |
| **3. Predict** | `model.predict([[7]])` |

| | |
|---|---|
| Regression | predict a number |
| Classification | predict a label |
| Clustering | find groups, no answers given |

Examples are a table: `[[1], [2], [3]]`, even with one column. And `thing.action()` is a **method**.

# Warm-up 1: list or array?

**Predict the output!**

```
import numpy as np

scores_list = [3, 5, 7]
scores = np.array([3, 5, 7])
print(scores_list + scores_list)
print(scores + scores)
print(scores * 10)
```



# Warm-up 1: list or array?

**Predict the output!**

```
import numpy as np

scores_list = [3, 5, 7]
scores = np.array([3, 5, 7])
print(scores_list + scores_list)
print(scores + scores)
print(scores * 10)
```

**Answer**

```
[3, 5, 7, 3, 5, 7]
[ 6 10 14]
[30 50 70]
```

Lists +: glue together. Arrays +: add item by item.



# Warm-up 2: the heat wave

**Predict the output!**

```
import numpy as np

temps = np.array([28, 33, 31, 26, 35])
print(temps > 30)
hot = temps[temps > 30]
print(hot, len(hot))
```



# Warm-up 2: the heat wave

**Predict the output!**

```
import numpy as np

temps = np.array([28, 33, 31, 26, 35])
print(temps > 30)
hot = temps[temps > 30]
print(hot, len(hot))
```

**Answer**

```
[False True True False True]
[33 31 35] 3
```

One True/False per day. The filter keeps only the True days.



# Warm-up 3: a 2-D array

### ❓ Two shops, three days of sales

```
import numpy as np

sales = np.array([[5, 8, 2],
                  [7, 1, 10]])
print(sales.shape)
print(sales[1, 2])
print(sales.mean(axis=1))
print(sales.max())
```

# Warm-up 3: a 2-D array

**Two shops, three days of sales**

```
import numpy as np

sales = np.array([[5, 8, 2],
                  [7, 1, 10]])
print(sales.shape)
print(sales[1, 2])
print(sales.mean(axis=1))
print(sales.max())
```

**Answer**

```
(2, 3)
10
[5. 6.]
10
```

2 rows, 3 columns. `axis=1`: across each row, so one average per shop.

# Warm-up 4: a tiny table

**Predict the output!**

```
import pandas as pd

films = {"title": ["Up", "Dune", "Coco"],
         "minutes": [96, 155, 105]}
df = pd.DataFrame(films)
print(df)
print(df["minutes"].sum())
```



# Warm-up 4: a tiny table

**Predict the output!**

```
import pandas as pd

films = {"title": ["Up", "Dune", "Coco"],
         "minutes": [96, 155, 105]}
df = pd.DataFrame(films)
print(df)
print(df["minutes"].sum())
```

OUTPUT

```
  title  minutes
0    Up       96
1  Dune      155
2  Coco      105
356
```

Keys become column names. The 0, 1, 2 on the left are the row numbers.



# Warm-up 5: which plot?

**Pick: line, bar, scatter or histogram**

1. My weight every week this year
2. How many students in each faculty
3. Height vs shoe size of 40 friends
4. The grades of all 300 students in the exam



# Warm-up 5: which plot?

**Pick: line, bar, scatter or histogram**

1. My weight every week this year
2. How many students in each faculty
3. Height vs shoe size of 40 friends
4. The grades of all 300 students in the exam

**Answer**

1 **line**: change over time
2 **bar**: compare groups
3 **scatter**: two numbers per person
4 **histogram**: how values spread



# Warm-up 6: water the plants

## Predict the output!

```
class Plant:
    def __init__(self, name):
        self.name = name
        self.height = 5

    def water(self):
        self.height = self.height + 2

mint = Plant("Mint")
basil = Plant("Basil")
mint.water()
mint.water()
basil.water()
print(mint.height, basil.height)
```

# Warm-up 6: water the plants

**Predict the output!**

```
class Plant:
    def __init__(self, name):
        self.name = name
        self.height = 5

    def water(self):
        self.height = self.height + 2

mint = Plant("Mint")
basil = Plant("Basil")
mint.water()
mint.water()
basil.water()
print(mint.height, basil.height)
```

**Answer**

9 7

Both start at 5. Mint was watered twice: $5 + 2 + 2 = 9$. Basil once: $5 + 2 = 7$. Each plant is its own object.

Topic 2

# Worked problems

Recitation 4 • Classes, NumPy, Pandas, plots & ML



TODAY

1 Recap and warm-up

**2 Worked problems**

3 Your turn

4 Exam corner

5 Solutions

# Problem 1: my step counter

**Your phone's weekly report**

Steps per day: 6500, 11200, 8300, 12900, 4100, 9800, 10200. How many km each day? Average and median? Days with 10,000+?

# Problem 1: my step counter

**Your phone's weekly report**

Steps per day: 6500, 11200, 8300, 12900, 4100, 9800, 10200. How many km each day? Average and median? Days with 10,000+?



**Plan in words first:**

1. Put the steps in a NumPy **array**
2. One step is about 0.8 m: multiply the **whole array**
3. Ask the array a **True/False question**

# Code it

step_counter.py

```
import numpy as np
steps = np.array([6500, 11200, 8300, 12900, 4100, 9800, 10200])
km = steps * 0.0008
print(km)
print(np.mean(steps), np.median(steps))
big_days = steps[steps >= 10000]
print(f"{len(big_days)} days with 10,000+ steps")
```

# Code it

step_counter.py

```python
import numpy as np
steps = np.array([6500, 11200, 8300, 12900, 4100, 9800, 10200])
km = steps * 0.0008
print(km)
print(np.mean(steps), np.median(steps))
big_days = steps[steps >= 10000]
print(f"{len(big_days)} days with 10,000+ steps")
```

OUTPUT

```
[ 5.2   8.96  6.64 10.32  3.28  7.84  8.16]
9000.0 9800.0
3 days with 10,000+ steps
```

# Code it

step_counter.py

```python
import numpy as np
steps = np.array([6500, 11200, 8300, 12900, 4100, 9800, 10200])
km = steps * 0.0008
print(km)
print(np.mean(steps), np.median(steps))
big_days = steps[steps >= 10000]
print(f"{len(big_days)} days with 10,000+ steps")
```

OUTPUT

```
[ 5.2   8.96  6.64 10.32  3.28  7.84  8.16]
9000.0 9800.0
3 days with 10,000+ steps
```

No loop needed! In Recitation 2 the same job took a `for` loop and two counters.

# What if...? Two weeks in one table

```
import numpy as np
weeks = np.array([[6500, 11200, 8300, 12900, 4100, 9800, 10200],
                  [7200, 9900, 10100, 8800, 5300, 12500, 11300]])
print(weeks.shape)
print(weeks.mean(axis=1))
print(weeks.mean(axis=0))
```

# What if...? Two weeks in one table

```
import numpy as np
weeks = np.array([[6500, 11200, 8300, 12900, 4100, 9800, 10200],
                  [7200, 9900, 10100, 8800, 5300, 12500, 11300]])
print(weeks.shape)
print(weeks.mean(axis=1))
print(weeks.mean(axis=0))
```

OUTPUT

```
(2, 7)
[9000. 9300.]
[ 6850. 10550.  9200. 10850.  4700. 11150. 10750.]
```

# What if...? Two weeks in one table

```
import numpy as np
weeks = np.array([[6500, 11200, 8300, 12900, 4100, 9800, 10200],
                  [7200, 9900, 10100, 8800, 5300, 12500, 11300]])
print(weeks.shape)
print(weeks.mean(axis=1))
print(weeks.mean(axis=0))
```

OUTPUT

```
(2, 7)
[9000. 9300.]
[ 6850. 10550.  9200. 10850.  4700. 11150. 10750.]
```

Rows: weeks, columns: days. `axis=1`: one average per **week** (across the row). `axis=0`: one per **day** (down the column).

# Problem 2: the campus library

loans.csv

```
student,faculty,books,late_days
Noa,Law,3,0
Omer,Medicine,5,2
Dana,Law,2,0
Yoni,Arts,4,7
Maya,Medicine,6,0
Itay,Arts,1,3
Shira,Law,4,1
Avi,Medicine,2,0
```

Made-up data. In Colab: upload the file first (folder icon).

**The librarian asks**

How many books are out? Which faculty has the most borrowers? Who is late, and by how much?

# Problem 2: the campus library

loans.csv

```
student,faculty,books,late_days
Noa,Law,3,0
Omer,Medicine,5,2
Dana,Law,2,0
Yoni,Arts,4,7
Maya,Medicine,6,0
Itay,Arts,1,3
Shira,Law,4,1
Avi,Medicine,2,0
```

Made-up data. In Colab: upload the file first (folder icon).

**The librarian asks**

How many books are out? Which faculty has the most borrowers? Who is late, and by how much?

**Plan in words:**

1. `read_csv`, then peek
2. A column, then `sum` and `mean`
3. `value_counts`, then a row filter

# Code it: load, peek, add up

library.py

```
import pandas as pd
df = pd.read_csv("loans.csv")
print(df.head(3))
print(df["books"].sum(), df["books"].mean())
```

# Code it: load, peek, add up

library.py

```
import pandas as pd
df = pd.read_csv("loans.csv")
print(df.head(3))
print(df["books"].sum(), df["books"].mean())
```

OUTPUT

```
   student   faculty  books  late_days
0      Noa       Law      3          0
1    Omer  Medicine      5          2
2    Dana       Law      2          0
27 3.375
```

# Code it: load, peek, add up

library.py

```python
import pandas as pd
df = pd.read_csv("loans.csv")
print(df.head(3))
print(df["books"].sum(), df["books"].mean())
```

OUTPUT

```
   student   faculty  books  late_days
0      Noa       Law      3          0
1    Omer  Medicine      5          2
2     Dana       Law      2          0
27 3.375
```

`head(3)`: only the first 3 rows. Empty brackets give 5.

27 books are out, 3.375 per student.

# Code it: count and filter

```
print(df["faculty"].value_counts())
```

```
late = df[df["late_days"] > 0]
print(late)
print(len(late), late["late_days"].mean())
```

# Code it: count and filter

```
print(df["faculty"].value_counts())
```

OUTPUT

```
faculty
Law         3
Medicine    3
Arts        2
Name: count, dtype: int64
```

A tie: Law and Medicine.

```
late = df[df["late_days"] > 0]
print(late)
print(len(late), late["late_days"].mean())
```

# Code it: count and filter

```
print(df["faculty"].value_counts())
```

OUTPUT

```
faculty
Law         3
Medicine    3
Arts        2
Name: count, dtype: int64
```

A tie: Law and Medicine.

```
late = df[df["late_days"] > 0]
print(late)
print(len(late), late["late_days"].mean())
```

OUTPUT

```
  student   faculty  books  late_days
1    Omer  Medicine      5          2
3    Yoni      Arts      4          7
5    Itay      Arts      1          3
6   Shira       Law      4          1
4 3.25
```

Rows keep their numbers.

# What if…? One faculty, and the full summary

```
law = df[df["faculty"] == "Law"]
print(law["books"].sum())
print(df.describe())
```

# What if…? One faculty, and the full summary

```
law = df[df["faculty"] == "Law"]
print(law["books"].sum())
print(df.describe())
```

Filter first, then ask the column.

`describe()` skips the text columns: only numbers get a summary.

OUTPUT

```
9
          books  late_days
count  8.000000   8.000000
mean   3.375000   1.625000
std    1.685018   2.445842
min    1.000000   0.000000
25%    2.000000   0.000000
50%    3.500000   0.500000
75%    4.250000   2.250000
max    6.000000   7.000000
```

# Problem 3: the ice cream stand

**The owner's notebook**

| Temperature (°C) | 18 | 21 | 24 | 27 | 30 | 33 |
|---|---|---|---|---|---|---|
| Cones sold | 40 | 55 | 62 | 80 | 91 | 104 |

Tomorrow will be 29°C. How many cones to prepare?

# Problem 3: the ice cream stand

**The owner's notebook**

| Temperature (°C) | 18 | 21 | 24 | 27 | 30 | 33 |
|---|---|---|---|---|---|---|
| Cones sold | 40 | 55 | 62 | 80 | 91 | 104 |

Tomorrow will be 29°C. How many cones to prepare?



**Plan in words:**

1. A number from a number: **regression**
2. Create, `fit`, `predict`
3. Draw the dots and the line to check

# Code it: create, fit, predict

ice_cream.py

```
from sklearn.linear_model import LinearRegression

temps = [[18], [21], [24], [27], [30], [33]]
cones = [40, 55, 62, 80, 91, 104]
model = LinearRegression()
model.fit(temps, cones)
print(model.predict([[29]]))
```

# Code it: create, fit, predict

ice_cream.py

```
from sklearn.linear_model import LinearRegression

temps = [[18], [21], [24], [27], [30], [33]]
cones = [40, 55, 62, 80, 91, 104]
model = LinearRegression()
model.fit(temps, cones)
print(model.predict([[29]]))
```

OUTPUT

```
[86.86666667]
```

# Code it: create, fit, predict

ice_cream.py

```
from sklearn.linear_model import LinearRegression

temps = [[18], [21], [24], [27], [30], [33]]
cones = [40, 55, 62, 80, 91, 104]
model = LinearRegression()
model.fit(temps, cones)
print(model.predict([[29]]))
```

OUTPUT

```
[86.86666667]
```

About 87 cones. Double brackets: the examples are a table with one column.

# Code it: create, fit, predict

ice_cream.py

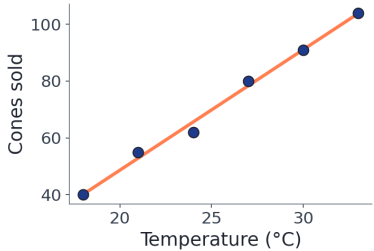
```
from sklearn.linear_model import LinearRegression

temps = [[18], [21], [24], [27], [30], [33]]
cones = [40, 55, 62, 80, 91, 104]
model = LinearRegression()
model.fit(temps, cones)
print(model.predict([[29]]))
```

OUTPUT

```
[86.86666667]
```

About 87 cones. Double brackets: the examples are a table with one column.




`plt.scatter(temps, cones)`, then `plt.plot(temps, model.predict(temps))`

# What if...? A very cold day

**What will the model say?**

```
print(model.predict([[0]]))
```

# What if...? A very cold day

**What will the model say?**

```
print(model.predict([[0]]))
```

OUTPUT

```
[-36.31428571]
```

**Minus 36 cones?!**

The model only saw 18 to 33°C. Far outside its examples, it just continues the line.



# What if...? A very cold day

**What will the model say?**

```
print(model.predict([[0]]))
```

OUTPUT

```
[-36.31428571]
```

**Minus 36 cones?!**

The model only saw 18 to 33°C. Far outside its examples, it just continues the line.



## A model is only as good as its examples.

# Problem 4: a strand of DNA

### The biology lab

DNA is a long text of 4 letters: A, T, G, C. **GC content** = the share of G and C letters (it tells how stable the strand is). The **complement** strand pairs A with T, and G with C.

# Problem 4: a strand of DNA

**The biology lab**

DNA is a long text of 4 letters: A, T, G, C. **GC content** = the share of G and C letters (it tells how stable the strand is). The **complement** strand pairs A with T, and G with C.



**Plan in words:**

1. A class `DNA` that keeps the letters
2. A method: count G and C in a loop (L06)
3. A method: swap each letter with a dictionary (L08)

# Code it: the class and its GC content

dna.py

```
class DNA:
    def __init__(self, sequence):
        self.sequence = sequence

    def gc_content(self):
        count = 0
        for base in self.sequence:
            if base == "G" or base == "C":
                count = count + 1
        return round(count / len(self.sequence), 2)
```

```
strand = DNA("ATGCGC")
print(strand.gc_content())
```

# Code it: the class and its GC content

dna.py

```
class DNA:
    def __init__(self, sequence):
        self.sequence = sequence

    def gc_content(self):
        count = 0
        for base in self.sequence:
            if base == "G" or base == "C":
                count = count + 1
        return round(count / len(self.sequence), 2)
```

```
strand = DNA("ATGCGC")
print(strand.gc_content())
```

OUTPUT

```
0.67
```

# Code it: the class and its GC content

dna.py

```
class DNA:
    def __init__(self, sequence):
        self.sequence = sequence

    def gc_content(self):
        count = 0
        for base in self.sequence:
            if base == "G" or base == "C":
                count = count + 1
        return round(count / len(self.sequence), 2)
```

```
strand = DNA("ATGCGC")
print(strand.gc_content())
```

OUTPUT

```
0.67
```

4 of the 6 letters are G or C: $4/6 \approx 0.67$. A loop and an **if**, inside a method.

# Code it: the complement strand

Add a second method to the same class:

```
    def complement(self):
        pairs = {"A": "T", "T": "A",
                 "G": "C", "C": "G"}
        result = ""
        for base in self.sequence:
            result = result + pairs[base]
        return result
```

```
strand = DNA("ATGCGC")
print(strand.complement())
```

# Code it: the complement strand

Add a second method to the same class:

```
    def complement(self):
        pairs = {"A": "T", "T": "A",
                 "G": "C", "C": "G"}
        result = ""
        for base in self.sequence:
            result = result + pairs[base]
        return result
```

```
strand = DNA("ATGCGC")
print(strand.complement())
```

OUTPUT

```
TACGCG
```

# Code it: the complement strand

Add a second method to the same class:

```
    def complement(self):
        pairs = {"A": "T", "T": "A",
                 "G": "C", "C": "G"}
        result = ""
        for base in self.sequence:
            result = result + pairs[base]
        return result
```

```
strand = DNA("ATGCGC")
print(strand.complement())
```

OUTPUT

```
TACGCG
```

A dictionary (L08) looks up each letter's partner. Old tools, inside a new box.

# What if...? A letter that is not A, T, G or C

**What happens?**

```
weird = DNA("ATGX")
print(weird.complement())
```

# What if...? A letter that is not A, T, G or C

**What happens?**

```
weird = DNA("ATGX")
print(weird.complement())
```

ERROR

```
KeyError: 'X'
```

**No pair for X**

`pairs["X"]` does not exist. Real lab files do have odd letters (N = unknown).

# What if…? A letter that is not A, T, G or C

**What happens?**

```
weird = DNA("ATGX")
print(weird.complement())
```

ERROR

```
KeyError: 'X'
```

**No pair for X**

`pairs["X"]` does not exist. Real lab files do have odd letters (`N` = unknown).

**Fix ideas:** check with `in` first, or catch it with `try` / `except KeyError` (L08).

## Topic 3

# Your turn

Recitation 4 • Classes, NumPy, Pandas, plots & ML



**TODAY**

1 Recap and warm-up

2 Worked problems

**3 Your turn**

4 Exam corner

5 Solutions

# Your turn: 11 exercises

| Exercise | Lecture | Level | Where |
|---|---|---|---|
| 1 Fix the VAT bug | L9 | ★ | in class |
| 2 Cheap snacks only | L9 | ★★ | in class |
| 3 A tiny photo | L9 | ★★ | at home |
| 4 Movie night table | L9 | ★ | in class |
| 5 My gym week | L9 | ★★ | in class |
| 6 Where did my money go? | L9 | ★★★ | at home |
| 7 Fix the rent chart | L10 | ★ | in class |
| 8 Pick the plot | L10 | ★★ | in class |
| 9 Predict the taxi fare | L10 | ★★ | at home |
| 10 Spam or not? | L10 | ★★★ | at home |
| 11 A shopping cart class | L9 | ★★ | at home |



Predict the output *before* you run. Stuck for 3 minutes? Ask!

# Exercise 1 ⭐: fix the VAT bug

EXERCISE 1

**This code has 2 bugs. Fix them.**

```
import numpy
prices = [12, 15, 9, 20]
with_vat = prices * 1.18
print(with_vat)
```

Run it first: `TypeError: can't multiply sequence by non-int of type 'float'`

**Wanted output:** 🗒️

OUTPUT

```
[14.16 17.7  10.62 23.6 ]
```

# Exercise 1 ⭐: fix the VAT bug

EXERCISE 1

**This code has 2 bugs. Fix them.**

```
import numpy
prices = [12, 15, 9, 20]
with_vat = prices * 1.18
print(with_vat)
```

Run it first: `TypeError: can't multiply sequence by non-int of type 'float'`

**Wanted output:**

OUTPUT

```
[14.16 17.7  10.62 23.6 ]
```

**Hint**

A list does not know math. Which toolbox does, and what is its nickname?

# Exercise 2 ⭐⭐: cheap snacks only

EXERCISE 2

fill in the yellow blanks

```
import numpy as np

snacks = np.        ([7, 12, 4, 9, 15, 6])
cheap = snacks[             ]
print(cheap)
print(np.       (cheap))
print(np.max(snacks) - np.min(snacks))
```

**Wanted output:** 🍪

OUTPUT

```
[7 4 9 6]
6.5
11
```

# Exercise 2 ⭐⭐: cheap snacks only

EXERCISE 2

fill in the yellow blanks

```
import numpy as np

snacks = np._____([7, 12, 4, 9, 15, 6])
cheap = snacks[_____]
print(cheap)
print(np._____(cheap))
print(np.max(snacks) - np.min(snacks))
```

**Wanted output:** 🍪

OUTPUT

```
[7 4 9 6]
6.5
11
```

**Hint**

Cheap: under 10 ILS. Line 2 is the **average** of the cheap ones.

# Exercise 3 ⭐⭐: a tiny photo

AT HOME 3

## Write it from scratch

A gray photo, 3 rows by 4 pixels (0 = black, 255 = white):

```
photo = np.array([[0, 50, 100, 150],
                  [200, 250, 255, 30],
                  [90, 120, 60, 15]])
```

Print its shape, the inverted photo, the crop of rows 0–1 and columns 1–2, and its average brightness.

## Wanted output: 📸

OUTPUT

```
(3, 4)
[[255 205 155 105]
 [ 55   5   0 225]
 [165 135 195 240]]
[[ 50 100]
 [250 255]]
110.0
```

# Exercise 3 ⭐⭐: a tiny photo

AT HOME 3

## 🖉 Write it from scratch

A gray photo, 3 rows by 4 pixels (0 = black, 255 = white):

```
photo = np.array([[0, 50, 100, 150],
                  [200, 250, 255, 30],
                  [90, 120, 60, 15]])
```

Print its shape, the inverted photo, the crop of rows 0–1 and columns 1–2, and its average brightness.

## Wanted output: 📸

OUTPUT

```
(3, 4)
[[255 205 155 105]
 [ 55   5   0 225]
 [165 135 195 240]]
[[ 50 100]
 [250 255]]
110.0
```

💡 Like Chelsea the cat in L09: `255 - photo`, then `[rows, cols]`.

# Exercise 4 ⭐: movie night table

EXERCISE 4

fill in the yellow blanks

```
import pandas as pd

movies = {"title": ["Fauda", "Shtisel", "Dune", "Coco"],
          "year": [2015, 2013, 2021, 2017],
          "rating": [8.2, 8.5, 8.0, 8.5]}
df = pd.                (movies)
print(df[          ].mean())
new = df[                    ]
print(new)
```

**Wanted output:** 🎥

OUTPUT

```
8.3
  title  year  rating
2  Dune  2021     8.0
3  Coco  2017     8.5
```

# Exercise 4 ⭐: movie night table

EXERCISE 4

fill in the yellow blanks

```
import pandas as pd

movies = {"title": ["Fauda", "Shtisel", "Dune", "Coco"],
          "year": [2015, 2013, 2021, 2017],
          "rating": [8.2, 8.5, 8.0, 8.5]}
df = pd.[          ](movies)
print(df[          ].mean())
new = df[                    ]
print(new)
```

**Wanted output:** 🎥

OUTPUT

```
8.3
  title  year  rating
2  Dune  2021     8.0
3  Coco  2017     8.5
```

**Hint**

new: only movies **after** 2015. Column names need quotes.

# Exercise 5 ⭐⭐: my gym week

EXERCISE 5

**gym.csv** (columns `day`, `activity`, `minutes`):

| Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|---|---|---|---|---|---|---|
| run | swim | run | yoga | run | swim | yoga |
| 30 | 45 | 25 | 60 | 35 | 30 | 50 |

fill in the blanks

```
import pandas as pd

df = pd.___________("gym.csv")
print(df["activity"].______________())
runs = df[df["activity"] ___ "run"]
print(runs["minutes"]._____())
print(df["minutes"].____())
```

**Wanted output:** 💪

OUTPUT

```
activity
run     3
swim    2
yoga    2
Name: count, dtype: int64
30.0
275
```

# Exercise 6 ⭐⭐⭐: where did my money go? AT HOME 6

## Write it from scratch with Pandas

This week (ILS): coffee 14, books 95, bus 6, coffee 16, pizza 52, bus 6, coffee 15.

Make a table with columns `item` and `ils`. Print the total, the average (1 decimal), how many of each item, and the purchases of 50+ ILS.

## Wanted output: 💰

OUTPUT

```
Total: 204 ILS
Average: 29.1 ILS
item
coffee    3
bus       2
books     1
pizza     1
Name: count, dtype: int64
    item  ils
1  books   95
4  pizza   52
```

# Exercise 6 ⭐⭐⭐: where did my money go? AT HOME 6

### Write it from scratch with Pandas

This week (ILS): coffee 14, books 95, bus 6, coffee 16, pizza 52, bus 6, coffee 15.

Make a table with columns `item` and `ils`. Print the total, the average (1 decimal), how many of each item, and the purchases of 50+ ILS.

💡 `{...:.1f}` from L03. Inside an f-string write `df['ils']`, with single quotes.

### Wanted output: 💰

OUTPUT

```
Total: 204 ILS
Average: 29.1 ILS
item
coffee    3
bus       2
books     1
pizza     1
Name: count, dtype: int64
    item  ils
1  books   95
4  pizza   52
```

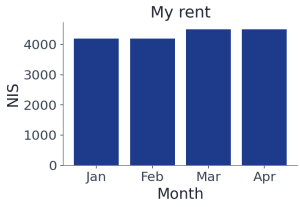
# Exercise 7 ⭐: fix the rent chart

EXERCISE 7

## 3 bugs. (April's rent was also 4500.)

```
import matplotlib as plt
months = ["Jan", "Feb", "Mar", "Apr"]
rent = [4200, 4200, 4500]
plt.bar(months, rent)
plt.title("My rent")
plt.xlable("Month")
plt.ylabel("ILS")
plt.show()
```

## Wanted output: 🏠




Your colors may differ.

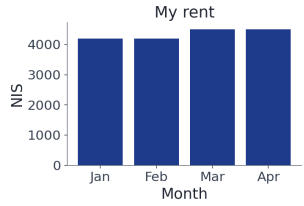
# Exercise 7 ⭐: fix the rent chart

EXERCISE 7

## 3 bugs. (April's rent was also 4500.)

```
import matplotlib as plt
months = ["Jan", "Feb", "Mar", "Apr"]
rent = [4200, 4200, 4500]
plt.bar(months, rent)
plt.title("My rent")
plt.xlable("Month")
plt.ylabel("ILS")
plt.show()
```

## Wanted output: 🏠




Your colors may differ. Fix one bug, run, read the **last line** of the error, repeat.

# Exercise 8 ⭐⭐: pick the plot

EXERCISE 8

fill in: plot, bar, scatter or hist

```
# 1. Shekel to dollar rate, every day for a month
plt.[      ](days, rate)
# 2. Votes for each of 5 parties
plt.[      ](parties, votes)
# 3. Hours on the phone vs hours of sleep, 50 people
plt.[      ](phone_hours, sleep_hours)
# 4. The ages of 1000 runners in a marathon
plt.[      ](ages)
```

# Exercise 8 ⭐⭐: pick the plot

EXERCISE 8

fill in: plot, bar, scatter or hist

```
# 1. Shekel to dollar rate, every day for a month
plt.[      ](days, rate)
# 2. Votes for each of 5 parties
plt.[      ](parties, votes)
# 3. Hours on the phone vs hours of sleep, 50 people
plt.[      ](phone_hours, sleep_hours)
# 4. The ages of 1000 runners in a marathon
plt.[      ](ages)
```

Over time? Groups? Two numbers per person? One long list of numbers?

# Exercise 9 ⭐⭐: predict the taxi fare

AT HOME 9

### Write it from scratch with scikit-learn

| km | 2 | 5 | 8 | 10 | 15 | 20 |
|---|---|---|---|---|---|---|
| fare (ILS) | 18 | 30 | 42 | 50 | 70 | 90 |

Train a `LinearRegression`. Predict 12 km and 30 km.

**Wanted output:** 🚙

OUTPUT

```
[ 58. 130.]
```

Bonus: the data is exactly 10 ILS + 4 ILS per km. Does the model find it?

# Exercise 9 ⭐⭐: predict the taxi fare

AT HOME 9

## Write it from scratch with scikit-learn

| km | 2 | 5 | 8 | 10 | 15 | 20 |
|---|---|---|---|---|---|---|
| fare (ILS) | 18 | 30 | 42 | 50 | 70 | 90 |

Train a `LinearRegression`. Predict 12 km and 30 km.

## Hint

Examples: `[[2], [5], ...]`. Two rides: `[[12], [30]]`.

**Wanted output:** 🚐

OUTPUT

```
[ 58. 130.]
```

Bonus: the data is exactly 10 ILS + 4 ILS per km. Does the model find it?

# Exercise 10 ⭐⭐⭐: spam or not?

AT HOME 10

## Write it from scratch: classification

Each email: [links, "!" marks]. **OK:** [0, 0], [1, 1], [0, 2], [2, 1]. **Spam:** [7, 9], [5, 12], [8, 6], [6, 10]. Train a `KNeighborsClassifier` (3 neighbors). Label [6, 8] and [1, 0], then [3, 4].

**Wanted output:** ✉

OUTPUT

```
['spam' 'ok']
['ok']
```

# Exercise 10 ★★★: spam or not?

AT HOME 10

## Write it from scratch: classification

Each email: [links, "!" marks]. **OK:** [0, 0], [1, 1], [0, 2], [2, 1]. **Spam:** [7, 9], [5, 12], [8, 6], [6, 10]. Train a `KNeighborsClassifier` (3 neighbors). Label [6, 8] and [1, 0], then [3, 4].

**Wanted output:**

OUTPUT

```
['spam' 'ok']
['ok']
```

### Hint

Like apple or banana in L10: a list of examples, a list of labels, then create, `fit`, `predict`.

# Exercise 11 ⭐⭐: a shopping cart class

EXERCISE 11

## Write it from scratch

A class `Cart`. It starts with an empty dictionary `items` (item → price).

Methods: `add(item, price)`, `total()` and `most_expensive()`.

Add a pita (6), hummus (14) and coffee (12), then print:

**Wanted output:** 🛒

OUTPUT

```
32 ILS
hummus
```

# Exercise 11 ⭐⭐: a shopping cart class

EXERCISE 11

### Write it from scratch

A class `Cart`. It starts with an empty dictionary `items` (item → price).

Methods: `add(item, price)`, `total()` and `most_expensive()`.

Add a pita (6), hummus (14) and coffee (12), then print:

**Wanted output:** 🛒

OUTPUT

```
32 ILS
hummus
```

### Hint

`self.items = {}` in `__init__`. For the max, the L06 recipe with `.items()`.

# Your home practice list

## Before the final project

- Exercises 3, 6, 9, 10 and 11 **at home**
- Any in-class exercise you did not finish
- Then check yourself against the solutions

Type the code yourself. Reading a solution is not practice.



## Topic 4

# Exam corner

Recitation 4 • Classes, NumPy, Pandas, plots & ML



**TODAY**

1 Recap and warm-up

2 Worked problems

3 Your turn

**4 Exam corner**

5 Solutions

# Exam style 1: trace the array

**What is printed?**

```
import numpy as np

a = np.array([2, 4, 6, 8])
b = a[a > 3] * 10
print(b)
print(len(b), b.mean())
```



# Exam style 1: trace the array

**What is printed?**

```
import numpy as np

a = np.array([2, 4, 6, 8])
b = a[a > 3] * 10
print(b)
print(len(b), b.mean())
```

**Answer**

```
[40 60 80]
3 60.0
```

First filter: `[4 6 8]`. Then times 10. The mean of 40, 60, 80 is 60.



Exam tip: write the array after every step. Arrays print with spaces, not commas.

# Exam style 2: no error, wrong answer

**Should print each student's average.**

```
import numpy as np
# rows: 3 students, columns: 2 tests
grades = np.array([[80, 90],
                   [60, 70],
                   [100, 80]])
print(grades.mean(axis=0))
```

OUTPUT

```
[80. 80.]
```

# Exam style 2: no error, wrong answer

**❓ Should print each student's average.**

```
import numpy as np
# rows: 3 students, columns: 2 tests
grades = np.array([[80, 90],
                   [60, 70],
                   [100, 80]])
print(grades.mean(axis=0))
```

OUTPUT

```
[80. 80.]
```

**Fix: `axis=1`, not `axis=0`**

```
print(grades.mean(axis=1))
```

OUTPUT

```
[85. 65. 90.]
```

3 students need 3 numbers. `axis=0` goes down each column: one average per **test**.

# Exam style 3: trace the table

**What is printed? Write every line.**

```
import pandas as pd
df = pd.DataFrame({
    "city": ["TLV", "Haifa", "TLV",
             "Eilat", "TLV"],
    "rain": [5, 12, 0, 1, 7]})
wet = df[df["rain"] > 4]
print(len(wet), wet["rain"].sum())
print(df["rain"].max(), df["rain"].median())
print(len(df[df["city"] == "TLV"]))
```



# Exam style 3: trace the table

**What is printed? Write every line.**

```
import pandas as pd
df = pd.DataFrame({
    "city": ["TLV", "Haifa", "TLV",
             "Eilat", "TLV"],
    "rain": [5, 12, 0, 1, 7]})
wet = df[df["rain"] > 4]
print(len(wet), wet["rain"].sum())
print(df["rain"].max(), df["rain"].median())
print(len(df[df["city"] == "TLV"]))
```

**Answer**

```
3 24
12 5.0
3
```

Rain over 4: 5, 12, 7. Sorted rain: 0, 1, **5**, 7, 12. TLV appears 3 times.



# Three new mistakes to avoid

```
prices = np.array([12, 15])
```

ERROR

```
NameError: name
'np' is not defined
```

`import` numpy `as` np first, in **every** new notebook.

```
df["Price"]
```

ERROR

```
KeyError: 'Price'
```

Names must match **exactly**: the column is `"price"`. Check with `df.head()`.

```
model.fit([1, 2], [60, 70])
```

ERROR

```
ValueError:
Expected 2D array,
got 1D array
instead:
```

Examples are a table: `[[1], [2]]`.

# Before the final project

**⚠ HW3 = final project: Data Analysis and Visualization**

- One CSV dataset you choose
- 3+ numbers and 2+ plots; your functions, a dict and a small class
- A 5-minute talk: what did you find?
- Submit on Moodle and present in the **last week** of the semester



- Start with `head()`: know your columns
- One question, one plot, one sentence

# Recap

**✓ Today you learned**

- Classes: `__init__` stores data in `self`, methods are the actions
- Arrays: math and True/False questions on every item at once
- Pandas: load, pick a column, filter rows, count, describe
- Pick the plot by the question; models: create, fit, predict



## Topic 5

# Solutions

Recitation 4 • Classes, NumPy, Pandas, plots & ML



**TODAY**

1 Recap and warm-up

2 Worked problems

3 Your turn

4 Exam corner

**5 Solutions**

# Solution 1: fix the VAT bug

SOLUTION

```
import numpy as np
prices = np.array([12, 15, 9, 20])
with_vat = prices * 1.18
print(with_vat)
```

OUTPUT

```
[14.16 17.7  10.62 23.6 ]
```

- `import numpy as np`: give it the nickname
- `np.array(...)`: a list cannot be multiplied by 1.18

A list times 2 would repeat it. A list times 1.18 is an error.

# Solution 2: cheap snacks only

SOLUTION

```
import numpy as np

snacks = np.array([7, 12, 4, 9, 15, 6])
cheap = snacks[snacks < 10]
print(cheap)
print(np.mean(cheap))
print(np.max(snacks) - np.min(snacks))
```

OUTPUT

```
[7 4 9 6]
6.5
11
```

$(7 + 4 + 9 + 6)/4 = 6.5$. The most expensive minus the cheapest: $15 - 4 = 11$.

# Solution 3: a tiny photo

SOLUTION

```
import numpy as np

photo = np.array([[0, 50, 100, 150],
                  [200, 250, 255, 30],
                  [90, 120, 60, 15]])
print(photo.shape)
print(255 - photo)
print(photo[0:2, 1:3])
print(photo.mean())
```

OUTPUT

```
(3, 4)
[[255 205 155 105]
 [ 55   5   0 225]
 [165 135 195 240]]
[[ 50 100]
 [250 255]]
110.0
```

Rows 0–1 means `0:2`: the stop is not in, as always.

# Solutions 4 and 5: movies and the gym

SOLUTION

## Exercise 4

```
df = pd.DataFrame(movies)
print(df["rating"].mean())
new = df[df["year"] > 2015]
print(new)
```

$(8.2 + 8.5 + 8.0 + 8.5)/4 = 8.3$. Dune and Coco are after 2015.

## Exercise 5

```
df = pd.read_csv("gym.csv")
print(df["activity"].value_counts())
runs = df[df["activity"] == "run"]
print(runs["minutes"].mean())
print(df["minutes"].sum())
```

Runs: $(30 + 25 + 35)/3 = 30$.

# Solution 6: where did my money go?

SOLUTION

```python
import pandas as pd

spending = {"item": ["coffee", "books", "bus",
                     "coffee", "pizza", "bus",
                     "coffee"],
            "ils": [14, 95, 6, 16, 52, 6, 15]}
df = pd.DataFrame(spending)
total = df["ils"].sum()
print(f"Total: {total} ILS")
print(f"Average: {df['ils'].mean():.1f} ILS")
print(df["item"].value_counts())
print(df[df["ils"] >= 50])
```

OUTPUT

```
Total: 204 ILS
Average: 29.1 ILS
item
coffee    3
bus       2
books     1
pizza     1
Name: count, dtype: int64
    item  ils
1  books   95
4  pizza   52
```

$204/7 = 29.14\ldots$ Coffee: only 45 ILS, but 3 times!

# Solution 7: fix the rent chart

SOLUTION

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr"]
rent = [4200, 4200, 4500, 4500]
plt.bar(months, rent)
plt.title("My rent")
plt.xlabel("Month")
plt.ylabel("ILS")
plt.show()
```

- `matplotlib.pyplot`: the plotting part
- One rent for every month: 4 values
- `xlabel`, not `xlable`

The errors, in order: `AttributeError`, `ValueError` (shape mismatch), `AttributeError`.

# Solution 8: pick the plot

SOLUTION

```
# 1. Shekel to dollar rate, every day for a month
plt.plot(days, rate)
# 2. Votes for each of 5 parties
plt.bar(parties, votes)
# 3. Hours on the phone vs hours of sleep, 50 people
plt.scatter(phone_hours, sleep_hours)
# 4. The ages of 1000 runners in a marathon
plt.hist(ages)
```

Over time: **line**. Groups: **bar**. Two numbers each: **scatter**. How values spread: **histogram**.

# Solution 9: predict the taxi fare

SOLUTION

```
from sklearn.linear_model import LinearRegression

km = [[2], [5], [8], [10], [15], [20]]
fare = [18, 30, 42, 50, 70, 90]
model = LinearRegression()
model.fit(km, fare)
print(model.predict([[12], [30]]))
```

OUTPUT

```
[ 58. 130.]
```

$10 + 4 \cdot 12 = 58$ and $10 + 4 \cdot 30 = 130$. The data sits exactly on a line, so the model finds it exactly.

30 km is outside the examples, but taxi fares really do keep growing in a line, so here it works.

# Solution 10: spam or not?

SOLUTION

```
from sklearn.neighbors import KNeighborsClassifier

emails = [[0, 0], [1, 1], [0, 2], [2, 1],
          [7, 9], [5, 12], [8, 6], [6, 10]]
labels = ["ok", "ok", "ok", "ok",
          "spam", "spam", "spam", "spam"]
model = KNeighborsClassifier(n_neighbors=3)
model.fit(emails, labels)
print(model.predict([[6, 8], [1, 0]]))
print(model.predict([[3, 4]]))
```

OUTPUT

```
['spam' 'ok']
['ok']
```

[3, 4] sits in the middle: its 3 nearest examples are all “ok”. Close calls are where models make mistakes.

3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md

# Solution 11: a shopping cart class

SOLUTION

```
class Cart:
    def __init__(self):
        self.items = {}
    def add(self, item, price):
        self.items[item] = price
    def total(self):
        return sum(self.items.values())
    def most_expensive(self):
        best = ""
        best_price = 0
        for item, price in self.items.items():
            if price > best_price:
                best = item
                best_price = price
        return best
```

```
cart = Cart()
cart.add("pita", 6)
cart.add("hummus", 14)
cart.add("coffee", 12)
print(cart.total(), "ILS")
print(cart.most_expensive())
```

OUTPUT

```
32 ILS
hummus
```